

# Rapport d'Audit de Sécurité

## 1. Résumé Exécutif

|                    |                                                                                                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cible :            | Laboratoire : Exploitation des vulnérabilités des API LLM (Injection de commandes)                                                                                  |
| Vulnérabilité :    | Injection de commandes système via les API de l'assistant LLM (*Command Injection*)                                                                                 |
| Niveau de Risque : | ÉLEVÉ (HIGH)                                                                                                                                                        |
| Impact :           | Exécution de code arbitraire à distance (RCE) sur le système sous-jacent, compromission des fichiers du système d'exploitation et suppression de données critiques. |

## 2. Description de la Vulnérabilité

L'application intègre un modèle de langage (LLM) connecté à une API d'abonnement à la newsletter (`functions.subscribe_to_newsletter`). En arrière-plan, le serveur traite l'adresse e-mail fournie en l'injectant directement dans une commande système (utilitaire de messagerie du système d'exploitation) sans assainissement ni validation rigoureuse des entrées.

En exploitant la substitution de commande Bash `$ ( . . . )` transmise via l'assistant, un attaquant peut forcer l'exécution de charges utiles arbitraires au niveau du système d'exploitation sous-jacent.

## 3. Preuve de Concept (PoC) & Étapes de Réalisation

- Étape 1 - Cartographie des API : Interrogation du LLM pour lister ses fonctions disponibles, révélant notamment l'outil `functions.subscribe_to_newsletter`.
- Étape 2 - Validation de l'interaction : Test initial d'abonnement avec l'adresse e-mail pointant vers l'Exploit Server externe, confirmant la bonne réception de la confirmation par le client de messagerie.
- Étape 3 - Test d'injection de commande (RCE) : Envoi d'une charge utile contenant `$ (whoami)` dans l'adresse e-mail. L'e-mail retourné a révélé le nom d'utilisateur système (*carlos*), confirmant l'exécution de la commande.

**Étape 4 - Action destructive et Résolution :** Contournement des filtres textuels du LLM par une formulation incitative pour exécuter la charge utile de suppression `$(rm /home/carlos/morale.txt)`, entraînant la suppression effective du fichier cible et la validation du laboratoire.

## 4. Recommandations de Remédiation

- **Validation stricte des formats d'entrées** : Implémenter une expression régulière (Regex) rigoureuse côté backend pour valider les adresses e-mail et rejeter systématiquement tout caractère de contrôle ou métacaractère Bash (`$`, ```, ``;`, ``|``, ``&``).
- **Éviter l'interprétation par le shell** : Remplacer les appels aux utilitaires système en ligne de commande par des bibliothèques natives sécurisées pour l'envoi d'e-mails, évitant ainsi tout recours à `system()` ou `exec()` avec des entrées utilisateur non filtrées.
- **Principe du moindre privilège** : Restreindre les privilèges du compte exécutant l'application web afin de limiter l'impact d'une éventuelle compromission système.

## 5. Conclusion

Ce laboratoire met en lumière les risques critiques associés au chaînage entre des agents conversationnels intelligents (LLM) et des backends vulnérables aux injections de commandes. Une sécurisation rigoureuse des interfaces de programmation (API) est indispensable pour prévenir l'exécution de code à distance.